In [10]:
```
# %pip install pandas numpy matplotlib seaborn scikit-learn scipy openpyxl p
```

In [28]:
```
#%pip install StandardScalar
```

```
ERROR: Could not find a version that satisfies the requirement StandardScala
r (from versions: none)
ERROR: No matching distribution found for StandardScalar
Note: you may need to restart the kernel to use updated packages.
```

In [ ]:
```
#
# Design n-Queens matrix having first Queen placed. Use backtracking to plac
# remaining Queens to generate the final n-queen's matrix.
```

In [66]:
```
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from sklearn.pipeline import make_pipeline

from sklearn.metrics import(precision_score,
accuracy_score, recall_score,
confusion_matrix,
classification_report)

data = pd.read_csv("diabetes.csv")
```

In [67]:
```
data.head()
```

Out[67]:

<style scoped> .dataframe tbody tr th:only-of-type { vertical-align: middle; } .dataframe tbody tr th { vertical-align: top; } .dataframe thead th { text-align: right; } </style> <thead> <th></th> <th>Pregnancies</th> <th>Glucose</th> <th>BloodPressure</th> <th>SkinThickness</th> <th>Insulin</th> <th>BMI</th> <th>Pedigree</th> <th>Age</th> <th>Outcome</th> </thead> <tbody> <th>0</th> <th>1</th> <th>2</th> <th>3</th> <th>4</th> </tbody> <tbody></tbody>

6	148	72	35	0	33.6	0.627	50	1
1	85	66	29	0	26.6	0.351	31	0
8	183	64	0	0	23.3	0.672	32	1
1	89	66	23	94	28.1	0.167	21	0
0	137	40	35	168	43.1	2.288	33	1

In [68]:
```
data.isnull().sum()
```

Out[68]:
```
Pregnancies      0
Glucose          0
BloodPressure    0
SkinThickness    0
Insulin          0
BMI              0
Pedigree         0
Age              0
Outcome          0
dtype: int64
```

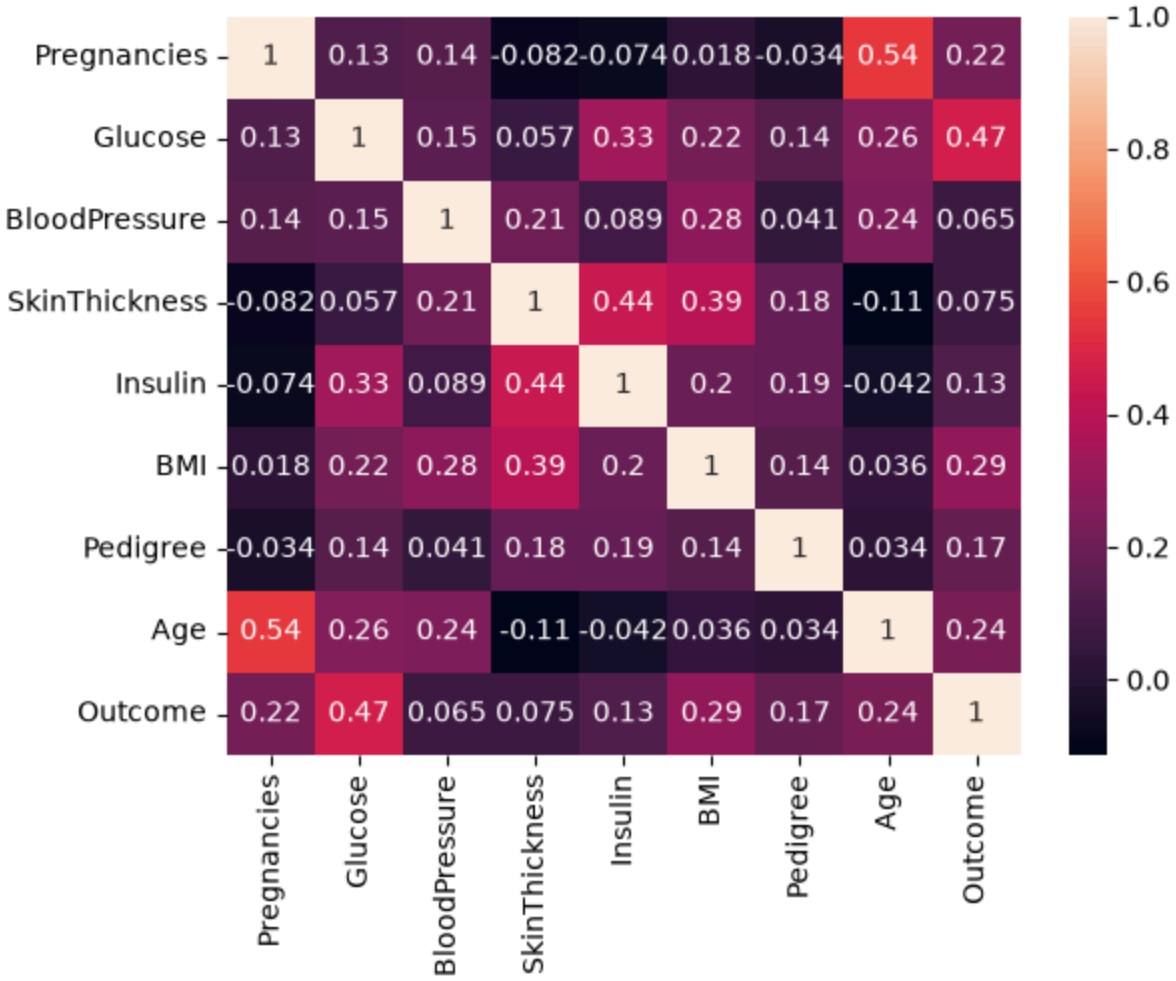
In [69]:
```
print(data.corr,["Outcome"])
```

```
<bound method DataFrame.corr of      Pregnancies  Glucose  BloodPressure  Sk
inThickness  Insulin   BMI  \
0              6      148             72             35        0  33.6
1              1       85             66             29        0  26.6
2              8      183             64              0        0  23.3
3              1       89             66             23       94  28.1
4              0      137             40             35      168  43.1
..           ...      ...            ...            ...      ...   ...
763           10      101             76             48      180  32.9
764            2      122             70             27        0  36.8
765            5      121             72             23      112  26.2
766            1      126             60              0        0  30.1
767            1       93             70             31        0  30.4

     Pedigree  Age  Outcome
0       0.627   50        1
1       0.351   31        0
2       0.672   32        1
3       0.167   21        0
4       2.288   33        1
..        ...  ...      ...
763     0.171   63        0
764     0.340   27        0
765     0.245   30        0
766     0.349   47        1
767     0.315   23        0

[768 rows x 9 columns]> ['Outcome']
```

In [70]:
```
sns.heatmap(data.corr(), annot=True)
plt.show()
```



In [71]:
```
X = data.drop('Outcome', axis=1)
y= data["Outcome"]
```

In [72]:
```
X_train,X_test,y_train,y_test = train_test_split(X,y,test_size = 0.3, random
```

In [73]:
```
knn = make_pipeline(StandardScaler(), KNeighborsClassifier(n_neighbors=7))
```

In [75]:
```
knn.fit(X_train,y_train)
```

Out[75]: <style>.sk-global { /* Definition of color scheme common for light and dark mode */ --sklearn-color-text: #000; --sklearn-color-text-muted: #666; --sklearn-color-line: gray; /* Definition of color scheme for unfitted estimators */ --sklearn-color-unfitted-level-0: #fff5e6; --sklearn-color-unfitted-level-1: #f6e4d2; --sklearn-color-unfitted-level-2: #ffe0b3; --sklearn-color-unfitted-level-3: chocolate; /* Definition of color scheme for fitted estimators */ --sklearn-color-fitted-level-0: #f0f8ff; --sklearn-color-fitted-level-1: #d4ebff; --sklearn-color-fitted-level-2: #b3dbfd; --sklearn-color-fitted-level-3: cornflowerblue; } .sk-global.light { /* Specific color for light theme */ --sklearn-color-text-on-default-background: black; --sklearn-color-background: white; --sklearn-color-border-box: black; --sklearn-color-icon: #696969; } .sk-global.dark { --sklearn-color-text-on-default-background: white; --sklearn-color-background: #111; --sklearn-color-border-box: white; --sklearn-color-icon: #878787; } .sk-global { color: var(--sklearn-color-text); } .sk-global pre { padding: 0; } .sk-global input.sk-hidden--visually { border: 0; clip-path: inset(100%); height: 1px; margin: -1px; overflow: hidden; padding: 0; position: absolute; width: 1px; } .sk-global div.sk-dashed-wrapped { border: 1px dashed var(--sklearn-color-line); margin: 0 0.4em 0.5em 0.4em; box-sizing: border-box; padding-bottom: 0.4em; background-color: var(--sklearn-color-background); } .sk-global div.sk-container { /* jupyter's `normalize.less` sets `[hidden] { display: none; }` but bootstrap.min.css set `[hidden] { display: none !important; }` so we also need the `!important` here to be able to override the default hidden behavior on the sphinx rendered scikit-learn.org. See: https://github.com/scikit-learn/scikit-learn/issues/21755 */ display: inline-block !important; position: relative; } .sk-global div.sk-text-repr-fallback { display: none; } div.sk-parallel-item, div.sk-serial, div.sk-item { /* draw centered vertical line to link estimators */ background-image: linear-gradient(var(--sklearn-color-text-on-default-background), var(--sklearn-color-text-on-default-background)); background-size: 2px 100%; background-repeat: no-repeat; background-position: center center; } /* Parallel-specific style estimator block */ .sk-global div.sk-parallel-item::after { content: ""; width: 100%; border-bottom: 2px solid var(--sklearn-color-text-on-default-background); flex-grow: 1; } .sk-global div.sk-parallel { display: flex; align-items: stretch; justify-content: center; background-color: var(--sklearn-color-background); position: relative; } .sk-global div.sk-parallel-item { display: flex; flex-direction: column; } .sk-global div.sk-parallel-item:first-child::after { align-self: flex-end; width: 50%; } .sk-global div.sk-parallel-item:last-child::after { align-self: flex-start; width: 50%; } .sk-global div.sk-parallel-item:only-child::after { width: 0; } /* Serial-specific style estimator block */ .sk-global div.sk-serial { display: flex; flex-direction: column; align-items: center; background-color: var(--sklearn-color-background); padding-right: 1em; padding-left: 1em; } /* Toggleable style: style used for estimator/Pipeline/ColumnTransformer box that is clickable and can be expanded/collapsed. - Pipeline and ColumnTransformer use this feature and define the default style - Estimators will overwrite some part of the style using the `sk-estimator` class */ /* Pipeline and ColumnTransformer style (default) */ .sk-global div.sk-toggleable { /* Default theme specific background. It is overwritten whether we have a specific estimator or a Pipeline/ColumnTransformer */ background-color: var(--

sklearn-color-background); } /* Toggleable label */ .sk-global label.sk-toggleable__label { cursor: pointer; display: flex; width: 100%; margin-bottom: 0; padding: 0.5em; box-sizing: border-box; text-align: center; align-items: center; justify-content: center; gap: 0.5em; } .sk-global label.sk-toggleable__label .caption { font-size: 0.6rem; font-weight: lighter; color: var(--sklearn-color-text-muted); } .sk-global label.sk-toggleable__label-arrow:before { /* Arrow on the left of the label */ content: "▸"; float: left; margin-right: 0.25em; color: var(--sklearn-color-icon); } .sk-global label.sk-toggleable__label-arrow:hover:before { color: var(--sklearn-color-text); } /* Toggleable content - dropdown */ .sk-global div.sk-toggleable__content { display: none; text-align: left; /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-toggleable__content.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); } .sk-global div.sk-toggleable__content pre { margin: 0.2em; border-radius: 0.25em; color: var(--sklearn-color-text); /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-toggleable__content.fitted pre { /* unfitted */ background-color: var(--sklearn-color-fitted-level-0); } .sk-global input.sk-toggleable__control:checked~div.sk-toggleable__content { /* Expand drop-down */ display: block; width: 100%; overflow: visible; } .sk-global input.sk-toggleable__control:checked~label.sk-toggleable__label-arrow:before { content: "▾"; } /* Pipeline/ColumnTransformer-specific style */ .sk-global div.sk-label input.sk-toggleable__control:checked~label.sk-toggleable__label { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-label.fitted input.sk-toggleable__control:checked~label.sk-toggleable__label { background-color: var(--sklearn-color-fitted-level-2); } /* Estimator-specific style */ /* Colorize estimator box */ .sk-global div.sk-estimator input.sk-toggleable__control:checked~label.sk-toggleable__label { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-2); } .sk-global div.sk-estimator.fitted input.sk-toggleable__control:checked~label.sk-toggleable__label { /* fitted */ background-color: var(--sklearn-color-fitted-level-2); } .sk-global div.sk-label label.sk-toggleable__label, .sk-global div.sk-label label { /* The background is the default theme color */ color: var(--sklearn-color-text-on-default-background); } /* On hover, darken the color of the background */ .sk-global div.sk-label:hover label.sk-toggleable__label { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); } /* Label box, darken color on hover, fitted */ .sk-global div.sk-label.fitted:hover label.sk-toggleable__label.fitted { color: var(--sklearn-color-text); background-color: var(--sklearn-color-fitted-level-2); } /* Estimator label */ .sk-global div.sk-label label { font-family: monospace; font-weight: bold; line-height: 1.2em; } .sk-global div.sk-label-container { text-align: center; } /* Estimator-specific */ .sk-global div.sk-estimator { font-family: monospace; border: 1px dotted var(--sklearn-color-border-box); border-radius: 0.25em; box-sizing: border-box; margin-bottom: 0.5em; /* unfitted */ background-color: var(--sklearn-color-unfitted-level-0); } .sk-global div.sk-estimator.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); } /* on hover */ .sk-global div.sk-estimator:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-2); }

.sk-global div.sk-estimator.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-2); } /* Specification for estimator info (e.g. "i" and "?") */ /* Common style for "i" and "?" */ .sk-estimator-doc-link, a:link.sk-estimator-doc-link, a:visited.sk-estimator-doc-link { float: right; font-size: smaller; line-height: 1em; font-family: monospace; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 1em; height: 1em; width: 1em; text-decoration: none !important; margin-left: 0.5em; text-align: center; /* unfitted */ border: var(--sklearn-color-unfitted-level-3) 1pt solid; color: var(--sklearn-color-unfitted-level-3); } .sk-estimator-doc-link.fitted, a:link.sk-estimator-doc-link.fitted, a:visited.sk-estimator-doc-link.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-3) 1pt solid; color: var(--sklearn-color-fitted-level-3); } /* On hover */ div.sk-estimator:hover .sk-estimator-doc-link:hover, .sk-estimator-doc-link:hover, div.sk-label-container:hover .sk-estimator-doc-link:hover, .sk-estimator-doc-link:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-3); border: var(--sklearn-color-fitted-level-0) 1pt solid; color: var(--sklearn-color-unfitted-level-0); text-decoration: none; } div.sk-estimator.fitted:hover .sk-estimator-doc-link.fitted:hover, .sk-estimator-doc-link.fitted:hover, div.sk-label-container:hover .sk-estimator-doc-link.fitted:hover, .sk-estimator-doc-link.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-3); border: var(--sklearn-color-fitted-level-0) 1pt solid; color: var(--sklearn-color-fitted-level-0); text-decoration: none; } /* Span, style for the box shown on hovering the info icon */ .sk-estimator-doc-link span { display: none; z-index: 9999; position: relative; font-weight: normal; right: .2ex; padding: .5ex; margin: .5ex; width: min-content; min-width: 20ex; max-width: 50ex; color: var(--sklearn-color-text); box-shadow: 2pt 2pt 4pt #999; /* unfitted */ background: var(--sklearn-color-unfitted-level-0); border: .5pt solid var(--sklearn-color-unfitted-level-3); } .sk-estimator-doc-link.fitted span { /* fitted */ background: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-3); } .sk-estimator-doc-link:hover span { display: block; } /* "?"-specific style due to the `` HTML tag */ .sk-global a.estimator_doc_link { float: right; font-size: 1rem; line-height: 1em; font-family: monospace; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 1rem; height: 1rem; width: 1rem; text-decoration: none; /* unfitted */ color: var(--sklearn-color-unfitted-level-1); border: var(--sklearn-color-unfitted-level-1) 1pt solid; } .sk-global a.estimator_doc_link.fitted { /* fitted */ background-color: var(--sklearn-color-fitted-level-0); border: var(--sklearn-color-fitted-level-1) 1pt solid; color: var(--sklearn-color-fitted-level-1); } /* On hover */ .sk-global a.estimator_doc_link:hover { /* unfitted */ background-color: var(--sklearn-color-unfitted-level-3); color: var(--sklearn-color-background); text-decoration: none; } .sk-global a.estimator_doc_link.fitted:hover { /* fitted */ background-color: var(--sklearn-color-fitted-level-3); } .sk-top-container.sk-global { /* pydata-sphinx-theme hides overflow, so scrolling is disabled. We need to set it to !important and add tabindex="0" in the HTML to allow keyboard-only users to navigate the display. */ overflow-x: scroll !important; max-width: 100%; } .estimator-table { font-family: monospace; } .estimator-table summary { padding: .5rem; cursor: pointer; } .estimator-table summary::marker { font-size: 0.7rem; }

.estimator-table details[open] { padding-left: 0.1rem; padding-right: 0.1rem; padding-bottom: 0.3rem; } .estimator-table .parameters-table { margin-left: auto !important; margin-right: auto !important; margin-top: 0; } .estimator-table .parameters-table tr:nth-child(odd) { background-color: #fff; } .estimator-table .parameters-table tr:nth-child(even) { background-color: #f6f6f6; } .estimator-table .parameters-table tr:hover td { background-color: #e0e0e0; } .estimator-table table :is(td, th) { border: 1px solid rgba(106, 105, 104, 0.232); } /* `table td`is set in notebook with right text-align. We need to overwrite it. */ .estimator-table table td.param { text-align: left; position: relative; padding: 0; } .user-set td { color:rgb(255, 94, 0); text-align: left !important; } .user-set td.value { color:rgb(255, 94, 0); background-color: transparent; } .default td, .estimator-table th { color: black; text-align: left !important; } .user-set td i, .default td i { color: black; } td.fitted-att-type { white-space: preserve nowrap; } /* Styles for parameter documentation links We need styling for visited so jupyter doesn't overwrite it */ a.param-doc-link, a.param-doc-link:link, a.param-doc-link:visited { text-decoration: underline dashed; text-underline-offset: .3em; color: inherit; display: block; padding: .5em; } @supports(anchor-name: --doc-link) { a.param-doc-link, a.param-doc-link:link, a.param-doc-link:visited { anchor-name: --doc-link; } } /* "hack" to make the entire area of the cell containing the link clickable */ a.param-doc-link::before { position: absolute; content: ""; inset: 0; } .param-doc-description { display: none; position: absolute; z-index: 9999; left: 0; padding: .5ex; margin-left: 1.5em; color: var(--sklearn-color-text); box-shadow: .3em .3em .4em #999; width: max-content; text-align: left; max-height: 10em; overflow-y: auto; /* unfitted */ background: var(--sklearn-color-unfitted-level-0); border: thin solid var(--sklearn-color-unfitted-level-3); } @supports(position-area: center right) { .param-doc-description { position-area: center right; position: fixed; margin-left: 0; } } /* Fitted state for parameter tooltips */ .fitted .param-doc-description { /* fitted */ background: var(--sklearn-color-fitted-level-0); border: thin solid var(--sklearn-color-fitted-level-3); } .param-doc-link:hover .param-doc-description { display: block; } .copy-paste-icon { background-image: url((background-repeat: no-repeat; background-size: 14px 14px; background-position: 0; display: inline-block; width: 14px; height: 14px; cursor: pointer; border: none; padding: 0; background-color: transparent; } .copy-paste-icon:focus-visible { outline: 1px solid currentColor; outline-offset: 1px; } .features { font-family: monospace; cursor: pointer; background-color: var(--sklearn-color-unfitted-level-0); border: 1px dotted var(--sklearn-color-border-box); border-radius: .20em; margin-bottom: 0.5em; font-size: inherit; /* Needed for jupyter */ } .features.fitted { background-color: var(--sklearn-color-fitted-level-0); } .features summary { cursor: pointer; display: flex; margin-bottom: 0; text-align: center; align-items: center; justify-content: center; gap: 0.5em; padding: .25em; } .features details[open] > summary { color: var(--sklearn-color-text); background-color: var(--sklearn-color-unfitted-level-2); border-radius: .20em 0 0 0; } .features.fitted details[open] > summary { background-color: var(--sklearn-color-fitted-level-2); border-radius: .20em 0 0 0; } .features details > summary .arrow::before {

content: "▸"; color: grey; } .features details[open] > summary .arrow::before { content: "▾"; } .features details:hover > summary { margin: 0; background-color: var(--sklearn-color-unfitted-level-2); } .features.fitted details:hover > summary { margin: 0; background-color: var(--sklearn-color-fitted-level-2); } .features .features-container { max-height: 10em; overflow: auto; scrollbar-width: thin; padding: .25em 0.1rem; background-color: var(--sklearn-color-unfitted-level-0); border-radius: 0 0 .5em .5em; } .features.fitted .features-container { background-color: var(--sklearn-color-fitted-level-0); } .features .image-container { block-size: 1em; inline-size: 1em; padding: 0; margin: 0%; display: flex; justify-content: center; align-items: center; } .features .copy-paste-icon { background-size: 1em 1em; width: 1em; height: 1em; filter: grayscale(100%) opacity(60%); } .features .features-container table { width: 100%; margin: 0.01em; } .features .features-container table tr:nth-child(odd) { background-color: #fff; } .features .features-container table tr:nth-child(even) { background-color: #f6f6f6; } .features .features-container table tr:hover { background-color: #e0e0e0; } .features .features-container table { table-layout: inherit; } .features .features-container table td { text-align: left; padding: 0 0.5em; border: 1px solid rgba(106, 105, 104, 0.232); white-space: nowrap; color: var(--sklearn-color-text); } .total_features { display: flex; justify-content: center; margin-top: 0.5em; } </style><body>

```
Pipeline(steps=[('standardscaler', StandardScaler()),
                ('kneighborsclassifier', KNeighborsClassifier(n_neighbors=7))])
```

**In a Jupyter environment, please rerun this cell to show the HTML representation or trust the notebook.
On GitHub, the HTML representation is unable to render, please try loading this page with nbviewer.org.**

<input class="sk-toggleable__control sk-hidden--visually sk-global" id="sk-estimator-id-7" type="checkbox" ><label for="sk-estimator-id-7" class="sk-toggleable__label fitted sk-toggleable__label-arrow">
Pipeline
?Documentation for PipelineiFitted
</label>
<details> <summary>Parameters</summary> <tbody> </tbody> <tbody></tbody>

<button type="button" class="copy-paste-icon" aria-label="Copy steps to clipboard" onclick="copyToClipboard('steps', this.parentElement.nextElementSibling)" > </button>	steps steps: list of tuples List of (r chained in sequential order. To be compatibl define `fit`. All non-last steps must also Estimator
<button type="button" class="copy-paste-icon" aria-label="Copy transform_input to clipboard" onclick="copyToClipboard('transform_input', this.parentElement.nextElementSibling)" > </button>	transform_input transform_input: list of :term:`metadata` parameters that should be trans to the step consuming it. This enables t (other than ``X``) to be transformed by t requires them. Requirement is defined via : For instance, this can be used to pass a vali can only set this if metadata routing ``sklearn.set_config(enable_metadata_rc
<button type="button" class="copy-paste-icon" aria-label="Copy memory to	memory memory: str or object with the joblib.Me to cache the fitted transformers of the pipeline. T

clipboard" onclick="copyToClipboard('memory', this.parentElement.nextElementSibling)" > </button>		it is a transformer. By default, no caching is p the caching directory. Enabling caching tri fitting. Therefore, the transformer instance g directly. Use the attribute ``named_steps`` pipeline. Caching the transformers is a See :ref:`sphx_glr_auto_examples_neighbors_
<button type="button" class="copy-paste-icon" aria-label="Copy verbose to clipboard" onclick="copyToClipboard('verbose', this.parentElement.nextElementSibling)" > </button>		verbose verbose: bool, default=False If

</details>

<details> <summary>Fitted attributes</summary> <tbody> <th>Name</th>

<th>Type</th> <th>Value</th> </tbody> <tbody></tbody>

classes_ classes_: ndarray of shape (n_classes,) The classes labels. Only exist if the last step of the pipeline is a classifier.	ndarray[int64] (2,)	
feature_names_in_ feature_names_in_: ndarray of shape (`n_features_in_`,) Names of features seen during :term:`fit`. Only defined if the underlying estimator exposes such an attribute when fit. .. versionadded:: 1.0	ndarray[object] (8,)	['Pregnancies','Glucose','BloodPressure',...,'BMI','Pedigree',
n_features_in_ n_features_in_: int Number of features seen during :term:`fit`. Only defined if the underlying first estimator in `steps` exposes such an attribute when fit. .. versionadded:: 0.24	int	

</details>

<input class="sk-toggleable__control sk-hidden--visually sk-global" id="sk-estimator-id-8" type="checkbox" ><label for="sk-estimator-id-8" class="sk-toggleable__label fitted sk-toggleable__label-arrow">

StandardScaler

?Documentation for StandardScaler

</label>

<details> <summary>Parameters</summary> <tbody> </tbody> <tbody></tbody>

<button type="button" class="copy-paste-icon" aria-label="Copy copy to clipboard" onclick="copyToClipboard('copy', this.parentElement.nextElementSibling)" > </button>	copy copy: bool, default=True If False, try to avoid a copy and do inplace scaling instead. This is not guaranteed to always work inplace; e.g. if the data is not a NumPy array or scipy.sparse CSR matrix, a copy may still be returned.	True
<button type="button" class="copy-paste-icon" aria-label="Copy with_mean to clipboard" onclick="copyToClipboard('with_mean', this.parentElement.nextElementSibling)" > </button>	with_mean with_mean: bool, default=True If True, center the data before scaling. This does not work (and will raise an exception) when attempted on sparse matrices, because centering them entails building a dense matrix which in common use cases is likely to be too large to fit in memory.	True
<button type="button" class="copy-paste-icon" aria-label="Copy with_std to clipboard" onclick="copyToClipboard('with_std', this.parentElement.nextElementSibling)" > </button>	with_std with_std: bool, default=True If True, scale the data to unit variance (or equivalently, unit standard deviation).	True

</details>

<details> <summary>Fitted attributes</summary> <tbody> <th>Name</th>

<th>Type</th> <th>Value</th> </tbody> <tbody></tbody>

feature_names_in_ feature_names_in_: ndarray of shape (`n_features_in_`,) Names of features seen during :term:`fit`. Defined only when `X` has feature names that are all strings. .. versionadded:: 1.0	ndarray[object] (8,)	['Pregnancies','Glucose','BloodPressure',...,'BMI','I
mean_ mean_: ndarray of shape (n_features,) or None The mean value for each feature in the training set. Equal to ``None`` when ``with_mean=False`` and ``with_std=False``.	ndarray[float64] (8,)	[3.73,120.85, 69.69,..., 31.

n_features_in_ n_features_in_: int Number of features seen during :term:`fit`. .. versionadded:: 0.24	int	
n_samples_seen_ n_samples_seen_: int or ndarray of shape (n_features,) The number of samples processed by the estimator for each feature. If there are no missing samples, the ``n_samples_seen`` will be an integer, otherwise it will be an array of dtype int. If `sample_weights` are used it will be a float (if no missing data) or an array of dtype float that sums the weights seen so far. Will be reset on new calls to fit, but increments across ``partial_fit`` calls.	float64	
scale_ scale_: ndarray of shape (n_features,) or None Per feature relative scaling of the data to achieve zero mean and unit variance. Generally this is calculated using `np.sqrt(var_)`. If a variance is zero, we can't achieve unit variance, and the data is left as-is, giving a scaling factor of 1. `scale_` is equal to `None` when `with_std=False`. .. versionadded:: 0.17 *scale_*	ndarray[float64] (8,)	[3.26,32.31,18.08,..., 7
var_ var_: ndarray of shape (n_features,) or None The variance for each feature in the training set. Used to compute `scale_`. Equal to ``None`` when ``with_mean=False`` and ``with_std=False``.	ndarray[float64] (8,)	[10.63,1043.9 , 326.8 ,..., 58.0

</details>

<details> <summary>

8 features

<button type="button" class="copy-paste-icon" title="Copy output features (max 100)" aria-label="Copy output features (max 100)" onclick=" event.stopPropagation(); event.preventDefault(); copyFeatureNamesToClipboard(this); " > </button>
</summary>
<tbody> </tbody> <tbody></tbody>

Pregnancies	
Glucose	
BloodPressure	
SkinThickness	
Insulin	
BMI	
Pedigree	
Age	

</details>
<input class="sk-toggleable__control sk-hidden--visually sk-global" id="sk-estimator-id-9" type="checkbox" ><label for="sk-estimator-id-9" class="sk-toggleable__label fitted sk-toggleable__label-arrow">
KNeighborsClassifier
?Documentation for KNeighborsClassifier
</label>
<details> <summary>Parameters</summary> <tbody> </tbody> <tbody></tbody>

<button type="button" class="copy-paste-icon" aria-label="Copy n_neighbors to clipboard" onclick="copyToClipboard('n_neighbors', this.parentElement.nextElementSibling)" > </button>	n_neighbors n_neighbors: int, default=5 N
<button type="button" class="copy-paste-icon" aria-label="Copy weights to clipboard" onclick="copyToClipboard('weights', this.parentElement.nextElementSibling)" > </button>	weights weights: {'uniform', 'distance'}, c Weight function used in prediction. Possibl weights. All points in each neighborhood a weight points by the inverse of their distance. query point will have a greater influence than n [callable] : a user-defined function which accepts a an array of the same shape containing the w entitled :ref:`sphx_glr_auto_examples_neighbor the impact of the `weights` par
<button type="button" class="copy-paste-icon" aria-label="Copy algorithm to clipboard" onclick="copyToClipboard('algorithm', this.parentElement.nextElementSibling)" > </button>	algorithm algorithm: {'auto', 'ball_tree', Algorithm used to compute the nearest ne :class:`BallTree` - 'kd_tree' will use :class:` force search. - 'auto' will attempt to decid based on the values passed to :meth:`fit` method will override the setting of
<button type="button" class="copy-paste-icon" aria-label="Copy leaf_size to	leaf_size leaf_size: int, default=30 Leaf s can affect the speed of the

clipboard" onclick="copyToClipboard('leaf_size', this.parentElement.nextElementSibling)" > </button>	memory required to store the tree. The optim
<button type="button" class="copy-paste-icon" aria-label="Copy p to clipboard" onclick="copyToClipboard('p', this.parentElement.nextElementSibling)" > </button>	p p: float, default=2 Power parameter for t is equivalent to using manhattan_distance (l1), For arbitrary p, minkowski_distance (l_p) is us
<button type="button" class="copy-paste-icon" aria-label="Copy metric to clipboard" onclick="copyToClipboard('metric', this.parentElement.nextElementSibling)" > </button>	metric metric: str or callable, default='minkows computation. Default is "minkowski", which resu when p = 2. See the docume <https://docs.scipy.org/doc/scipy/reference/spatial listed in :class:`~sklearn.metric metric values. If metric is "precom matrix and must be square during fi which case only "nonzero" elements may be co is a callable function, it takes two arrays represent return one value indicating the distance Scipy's metrics, but is less efficient tha
<button type="button" class="copy-paste-icon" aria-label="Copy metric_params to clipboard" onclick="copyToClipboard('metric_params', this.parentElement.nextElementSibling)" > </button>	metric_params metric_params: dict, defa
<button type="button" class="copy-paste-icon" aria-label="Copy n_jobs to clipboard" onclick="copyToClipboard('n_jobs', this.parentElement.nextElementSibling)" > </button>	n_jobs n_jobs: int, default=None <br: neighbors search. ``None`` means 1 un context. ``-1`` means using all processors. S more details.<

</details>

<details> <summary>Fitted attributes</summary> <tbody> <th>Name</th>

<th>Type</th> <th>Value</th> </tbody> <tbody></tbody>

classes_ classes_: array of shape (n_classes,) Class labels known to the classifier	ndarray[int64] (2,)	[0,1]
effective_metric_ effective_metric_: str or callble The distance metric used. It will be same as the `metric` parameter or a synonym of it, e.g. 'euclidean' if the `metric` parameter set to 'minkowski' and `p` parameter set to 2.	str	'eu...an'
effective_metric_params_ effective_metric_params_: dict Additional keyword arguments for the metric function. For most metrics will be same with `metric_params` parameter, but may also contain the `p` parameter value if the `effective_metric_` attribute is set to 'minkowski'.	dict	{}
n_features_in_ n_features_in_: int Number of features seen during :term:`fit`. .. versionadded:: 0.24	int	8
n_samples_fit_ n_samples_fit_: int Number of samples in the fitted data.	int	537

outputs_2d_ outputs_2d_: bool False when `y`'s shape is (n_samples,) or (n_samples, 1) during fit otherwise True.	bool	False

</details>

<script>/* Authors: The scikit-learn developers SPDX-License-Identifier: BSD-3-Clause */ function copyToClipboard(text, element) { // Get the parameter prefix from the closest toggleable content const toggleableContent = element.closest('.sk-toggleable__content'); const paramPrefix = toggleableContent ? toggleableContent.dataset.paramPrefix : ''; const fullParamName = paramPrefix ? `$paramPrefix${text}` : text; const originalStyle = element.style; const computedStyle = window.getComputedStyle(element); const originalWidth = computedStyle.width; const originalHTML = element.innerHTML.replace('Copied!', ''); navigator.clipboard.writeText(fullParamName) .then(() => { element.style.width = originalWidth; element.style.color = 'green'; element.innerHTML = "Copied!"; setTimeout(() => { element.innerHTML = originalHTML; element.style = originalStyle; }, 2000); }) .catch(err => { console.error('Failed to copy:', err); element.style.color = 'red'; element.innerHTML = "Failed!"; setTimeout(() => { element.innerHTML = originalHTML; element.style = originalStyle; }, 2000); }); return false; } document.querySelectorAll('.copy-paste-icon').forEach(function(element) { const toggleableContent = element.closest('.sk-toggleable__content'); const paramPrefix = toggleableContent ? toggleableContent.dataset.paramPrefix : ''; const parent = element.parentElement; if (!parent || !parent.nextElementSibling) { console.warn('Expected copy-paste icon is missing from the DOM structure'); return; } const paramName = element.parentElement.nextElementSibling .textContent.trim().split(' ')[0]; const fullParamName = paramPrefix ? `$paramPrefix${paramName}` : paramName; element.setAttribute('title', fullParamName); }); /** * Copy the list of feature names formatted as a Python list. * * @param {HTMLElement} element - The copy button inside a `.features` block; its siblings * contain a `details` element and a table containing feature named. * @returns {boolean} Always returns `false` so callers can prevent the default click behavior. */ function copyFeatureNamesToClipboard(element) { var detailsElem = element.closest('.features').querySelector('details'); var wasOpen = detailsElem.open; detailsElem.open = true; var content = element.closest('.features').querySelector('tbody') .innerText.trim(); if (!wasOpen) detailsElem.open = false; const rows = content.split('\n').map(row => ` " $row$ " `); $const formattedText = `[\n{rows.join(',\n')},\n]`; const originalHTML = element.innerHTML.replace('✓ ', ''); const originalStyle = element.style; const copyMark = document.createElement('span'); copyMark.innerHTML = '✓ '; copyMark.style.color = 'blue'; copyMark.style.fontSize = '1em'; navigator.clipboard.writeText(formattedText) .then(() => { element.style.display = 'none'; element.parentElement.appendChild(copyMark); setTimeout(() => { copyMark.remove(); element.innerHTML = originalHTML; element.style = originalStyle; }, 1000); }) .catch(err => { console.error('Failed to copy:', err); element.style.color = 'orange'; element.innerHTML = "Failed!"; setTimeout(() => { element.innerHTML = originalHTML;

element.style = originalStyle; }, 1000); }); return false; } /** * Adapted from Skrub * https://github.com/skrub-data/skrub/blob/403466d1d5d4dc76a7ef569b3f8228db59a31dc3/skrub/_reporting/_data/te * @returns "light" or "dark" */ function detectTheme(element) { const body = document.querySelector('body'); // Check VSCode theme const themeKindAttr = body.getAttribute('data-vscode-theme-kind'); const themeNameAttr = body.getAttribute('data-vscode-theme-name'); if (themeKindAttr && themeNameAttr) { const themeKind = themeKindAttr.toLowerCase(); const themeName = themeNameAttr.toLowerCase(); if (themeKind.includes("dark") || themeName.includes("dark")) { return "dark"; } if (themeKind.includes("light") || themeName.includes("light")) { return "light"; } } // Check Jupyter theme if (body.getAttribute('data-jp-theme-light') === 'false') { return 'dark'; } else if (body.getAttribute('data-jp-theme-light') === 'true') { return 'light'; } // Guess based on a parent element's color const color = window.getComputedStyle(element.parentNode, null).getPropertyValue('color'); const match = color.match(/^rgb\s* \s * (\d+)\s*, \s * (\d+)\s*, \s * (\d+)\s*\s*$/i); if (match) { const [r, g, b] = [parseFloat(match[1]), parseFloat(match[2]), parseFloat(match[3])]; // https://en.wikipedia.org/wiki/HSL_and_HSV#Lightness const luma = 0.299 * r + 0.587 * g + 0.114 * b; if (luma > 180) { // If the text is very bright we have a dark theme return 'dark'; } if (luma < 75) { // If the text is very dark we have a light theme return 'light'; } // Otherwise fall back to the next heuristic. } // Fallback to system preference return window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light'; } function forceTheme(elementId) { const estimatorElement = document.querySelector(`#${elementId}`); if (estimatorElement === null) { console.error(`Element with id ${elementId} not found.`); } else { const theme = detectTheme(estimatorElement); estimatorElement.classList.add(theme); } } forceTheme('sk-container-id-3');</script></body>

In [76]:
```
y_pred = knn.predict(X_test)
```

In [77]:
```
cm = confusion_matrix(y_test, y_pred)
accuracy = accuracy_score(y_test, y_pred)
error_rate = 1- accuracy
precision = precision_score(y_test, y_pred)
recall = recall_score(y_test, y_pred)

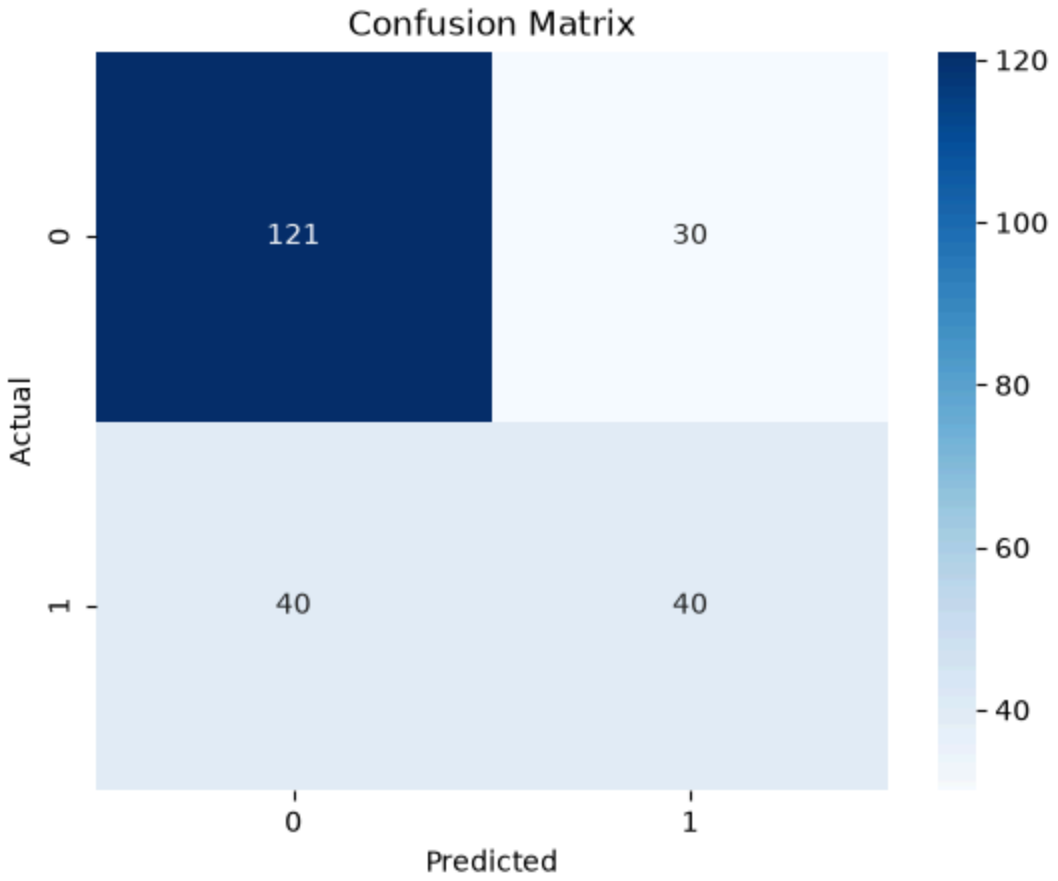

print("Confusion matrix:", cm)
print("accuracy:",accuracy)
print("Precision: ", precision)
print("Recall: ", recall)
print("classification reports: ",classification_report(y_test, y_pred))
```

```
Confusion matrix: [[121  30]
 [ 40  40]]
accuracy: 0.696969696969697
Precision:  0.5714285714285714
Recall:  0.5
classification reports:                precision    recall  f1-score   suppo
rt

           0       0.75      0.80      0.78       151
           1       0.57      0.50      0.53        80

    accuracy                           0.70       231
   macro avg       0.66      0.65      0.65       231
weighted avg       0.69      0.70      0.69       231
```

In [78]:
```
sns.heatmap(cm , annot=True,fmt='d', cmap="Blues")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```



In [ ]:
```

```